



Universidad De Cartagena

Facultad De Ingenieria

***IMPLEMENTACIÓN Y ENTRENAMIENTO DEL PERCEPTRÓN SIMPLE BIPOLAR EN
PROBLEMAS DE CLASIFICACIÓN LINEAL: SIMULACIÓN DE CONDICIONAMIENTO
INSTRUMENTAL Y DIAGNÓSTICO MÉDICO***

Presentado por:

Dago David Palmera Navarro - 0222321003

Julián David Cámara Padilla - 0222320016

Docente:

Manuel Alejandro Ospina Alarcón

Ingenieria De Sistemas

Semestre VII

Inteligencia Artificial

Cartagena de Indias, Septiembre 13 del 2026

TABLA DE CONTENIDO

INTRODUCCIÓN:	3
OBJETIVOS:	4
• Objetivo General:	4
• Objetivos Específicos:	4
FUNDAMENTACIÓN TEÓRICA:	5
DESARROLLO DE LA ACTIVIDAD – CASO DE ESTUDIO 1: CONDICIONAMIENTO DE PALOMA:	8
• Planteamiento del Problema y Variables de Entrada/Salida.	8
• Tabla de Verdad Bipolar (8 Patrones).	9
• Condiciones Iniciales (<i>pesos</i> W^0 y <i>sesgo</i> b^0).	9
• Curva de Aprendizaje (Evolución del Error Época tras Época).	10
• Parámetros Calibrados Finales (W^* y b^*).	20
• Ecuación Analítica del Hiperplano y Gráfica 3D de los Patrones.	21
DESARROLLO DE LA ACTIVIDAD – CASO DE ESTUDIO 2: DIAGNÓSTICO MÉDICO POR SÍNTOMAS:	22
• Planteamiento del Problema y Criterio de Decisión Clínica.	22
• Tabla de Verdad Bipolar (8 Patrones de Salud/Enfermedad).	23
• Condiciones Iniciales (W^0 y b^0).	24
• Curva de Aprendizaje (Evolución del Error).	24
• Parámetros Calibrados Finales (W^* y b^*).	35
• Ecuación Analítica del Hiperplano y Gráfica 3D de los Patrones.	36
• Estudio Comparativo de Sensibilidad a las Condiciones Iniciales (Convergencia en 3, 2 y 1 Épocas).	37
ANÁLISIS Y DISCUSIÓN DE RESULTADOS:	39
CONCLUSIONES:	41

INTRODUCCIÓN:

En el campo de la Inteligencia Artificial el aprendizaje automático, las Redes Neuronales Artificiales (RNA) constituyen uno de los programas más representativos de la computación bioinspirada, orientados al reconocimiento de patrones, la aproximación funcional y la toma autónoma de decisiones. Entre los modelos fundacionales destaca el Perceptrón Simple (PS), propuesto por Frank Rosenblatt en 1958, el cual formaliza una arquitectura monocapa capaz de ajustar adaptativamente sus parámetros sinápticos mediante un esquema de aprendizaje supervisado.

La capacidad operativa del Perceptrón Simple radica en su habilidad para establecer fronteras lineales de clasificación –representadas geométricamente como hiperplanos de decisión– que dividen el espacio de estados en regiones disjuntas. En este contexto, el uso de representaciones estrictamente bipolares $\{-1, +1\}$ y funciones de activación escalón simétricas (hardlims) aporta una ventaja matemática sustancial sobre esquemas binarios convencionales, garantizando que cada componente del vector de entrada contribuya activamente a la corrección del error sin inducir estancamientos durante la actualización por regla delta.

El presente informe consolida el diseño, entrenamiento y simulación computacional del Perceptrón Simple aplicado a dos casos de estudio con entradas tridimensionales: en primer lugar, la modelación de un experimento biológico de condicionamiento instrumental operante en una paloma; y en segundo lugar, un clasificador de diagnóstico médico preliminar basado en la concurrencia de sintomatologías clínicas. A lo largo del documento se detallan las tablas de patrones, la dinámica de calibración de pesos y sesgos, la reducción del error global a lo largo de las épocas y la visualización tridimensional del plano de separación lineal resultante.

OBJETIVOS:

- Objetivo General:

Diseñar, implementar y evaluar un modelo de red neuronal artificial del tipo Perceptrón Simple (PS) operando en el dominio bipolar simétrico $\{-1, +1\}$, con el fin de resolver problemas de clasificación binaria linealmente separables a través de la simulación de toma de decisiones en condicionamiento instrumental y diagnóstico médico por patrones clínicos.

- Objetivos Específicos:

1. **Analizar** los fundamentos matemáticos, biológicos y geométricos que rigen la operación de la neurona artificial y el hiperplano delimitador de clases en espacios vectoriales R^3 .
2. **Formular y parametrizar** las tablas de verdad y matrices de entrenamiento bajo representación bipolar $\{-1, +1\}$ para los casos de condicionamiento de la paloma y diagnóstico de sintomatología clínica.
3. **Aplicar** el algoritmo de entrenamiento supervisado mediante la regla delta y la función de activación bipolar hardlims, examinando la dinámica de corrección de pesos sinápticos (W) y sesgo (b) época a época.
4. **Evaluar** la curva de convergencia del error global del sistema hasta alcanzar el estado de cero errores ($E_{global} = 0$), contrastando la sensibilidad del algoritmo frente a las condiciones iniciales asignadas.
5. **Representar geométricamente** en tres dimensiones el hiperplano de separación obtenido, verificando la partición del espacio vectorial y la correcta clasificación de los ocho estados posibles para cada caso de estudio.

FUNDAMENTACIÓN TEÓRICA:

- **Antecedentes e Inspiración Biológica de la Neurona Artificial**

Las Redes Neuronales Artificiales (RNA) surgen como una abstracción bioinspirada de los mecanismos de procesamiento de información del cerebro humano. En el sistema nervioso biológico, la neurona constituye la unidad elemental de cómputo; esta recibe impulsos electroquímicos a través de un árbol de ramificaciones denominado **dendritas**, acumula la diferencia de potencial dentro del cuerpo celular (*soma*) y, cuando la estimulación sobrepasa cierto umbral crítico de despolarización, dispara un potencial de acción que se propaga a lo largo del axón hacia otras células mediante conexiones especializadas llamadas sinapsis.

En 1943, Warren McCulloch y Walter Pitts desarrollaron el primer modelo matemático formal de una neurona. En dicha propuesta, demostraron que una unidad de umbral binario podía computar funciones lógicas fundamentales (como AND, OR y NOT). No obstante, el modelo de conexión como el umbral debían establecerse de forma manual y estática, impidiendo que la neurona aprendiera a partir de la experiencia empírica.

- **Aprendizaje Hebbiano y Plasticidad Sináptica**

La base teórica que explica cómo un sistema neuronal modifica físicamente sus enlaces fue postulada por Donald Hebb en 1949 a través de la conocida **Ley de Hebb** (*Hebbian Learning*). Hebb planteó que cuando el axón de una neurona A se encuentra lo suficientemente cerca de una neurona B para excitarla y participa repetidamente en su disparo, ocurre un proceso de crecimiento o cambio metabólico en una o ambas células que incrementa la fuerza sináptica entre ellas.

Este principio, sintetizado modernamente bajo el aforismo “*cells that fire together, wire together*”, introdujo formalmente el concepto de peso variable como parámetro dinámico de memoria: aprender no consiste en almacenar datos en una dirección fija de memoria, sino en ajustar, reforzar o atenuar la conductancia o influencia mutua entre nodos interconectados.

- **El Percetrón Simple de Rosenblatt**

En 1958, Frank Rosenblatt integró la concepción de la neurona formal con el principio de *plasticidad de Hebb*, dando origen al **Percetrón Simple (PS)**. A diferencia de sus predecesores, Rosenblatt dotó al modelo de una regla algorítmica de convergencia capaz de ajustar iterativamente los parámetros del sistema mediante supervisión externa, convirtiéndose en el primer modelo de red neuronal capaz de aprender a clasificar patrones de forma autónoma.

Anatomía Matemática del Perceptrón:

1. **Vector de entrada (X):** Representa las características observables del patrón presentado al sistema en un espacio $n - dimensional$: $X = [\epsilon R^n [x_1, x_2, \dots, x_n]^T$
2. **Vector de Pesos Sinápticos (W):** Modela la intensidad de cada enlace sináptico: $X = [[x_1, x_2, \dots, x_n]^T \in R^n$. Un peso positivo ($w_j > 0$) denota una conexión

excitatoria, mientras que un peso negativo ($w_j < 0$) ejerce un efecto inhibitorio sobre el disparo.

3. **Sesgo o Umbral (*bias*, b):** Parámetro escalar independiente que confiere un grado de libertad adicional de decisión a la red: $b = -\theta$. Permite trasladar espacialmente la función de decisión respecto al origen de coordenadas, asegurando que la neurona pueda producir respuestas arbitrarias incluso cuando todas las componentes de entrada son nulas ($\mathbf{X} = \mathbf{0}$).
4. **Combinación Lineal (Salida Neta, a):** Agrega las entradas ponderadas junto con el término de sesgo: $a = \sum_{j=1}^n w_j x_j + b = \mathbf{W}^T \mathbf{X} + b$.
5. **Función de Activación ($f(a)$):** Mapea la suma neta escalar a al espacio de decisión de salida $y = f(a)$.

- **Geometría de la Decisión y Separabilidad Lineal**

El Perceptrón Simple opera como un discriminador lineal que particiona el espacio de estados R^n en dos semiespacios disjuntos. La frontera geométrica donde se produce la transición de estado (cambio de clase) corresponde al lugar donde la excitación neta se anula:

$$\mathbf{W}^T \mathbf{X} + b = 0$$

Para un espacio tridimensional R^3 , como el abordado en los casos prácticos de este estudio con entradas (x_1, x_2, x_3) , dicha frontera conforma un **hiperplano de decisión bidimensional** definido analíticamente por:

$$w_1 x_1 + w_2 x_2 + w_3 x_3 + b = 0$$

- El vector de pesos $\mathbf{W}^* = [w_1^*, w_2^*, w_3^*]^T$ actúa como el **vector normal ortogonal** que define la orientación del plano.
- La magnitud y signo del sesgo b^* determinan el **desplazamiento euclidiano** perpendicular al plano respecto al origen $(0, 0, 0)$.

Un conjunto de datos se denomina linealmente separable si existe al menos un hiperplano capaz de aislar a todos los puntos de la clase positiva ($y = +1$), en uno de sus semiespacios ($a \geq 0$), y a todos los puntos de la clase negativa ($y = -1$) en el semiespacio opuesto ($a < 0$).

- **Justificación Técnica del Dominio Bipolar frente al Unipolar**

En el modelado del Perceptrón Simple resulta crítico distinguir entre dos dominios de representación discreta:

1. **Dominio Unipolar $\{0, 1\}$:** Utiliza la función escalón clásica $\text{hardlim}(a)$:

$$f(a) = \begin{cases} 1 & \text{si } a \geq 0 \\ 0 & \text{si } a < 0 \end{cases}$$

2. **Dominio Bipolar $\{-1, +1\}$:** Utiliza la función escalón simétrica $\text{hardlim}(a)$:

$$f(a) = \begin{cases} +1 & \text{si } a \geq 0 \\ -1 & \text{si } a < 0 \end{cases}$$

Fundamento de la Selección Bipolar:

En esquemas unipolares $\{0, 1\}$, cuando una variable de entrada adopta el valor de inactividad ($x_j = 0$), el producto que gobierna el término de corrección en la regla de aprendizaje delta se extingue idénticamente: $\Delta w_j = e \cdot x_j = e \cdot (0) = 0$. Bajo esta condición, el peso sináptico w_j queda completamente congelado durante esa iteración, independientemente de que la neurona haya emitido una respuesta errónea.

Por el contrario, bajo la convención estrictamente **bipolar** $\{-1, +1\}$, toda variable de entrada aporta de manera permanente un valor no nulo con signo específico ($+1$ ó -1). Esto garantiza que **todas las componentes del vector de pesos se actualicen de manera solidaria en cada paso de aprendizaje**, orientando activamente el vector normal W y evitando estancamientos numéricos, lo cual incrementa notablemente en la tasa de convergencia del sistema.

- **Algoritmo de Aprendizaje Supervisado y Regla Delta**

El entrenamiento del Percetrón Simple se rige por un esquema supervisado basado en pares ordenados $(X_k, y_{d,k})$, donde X_k es el vector del patrón k –ésimo y $y_{d,k} \in \{-1, +1\}$ representa la salida deseada (*target*).

Dinámica de Corrección de Errores:

El error de clasificación individual se calcula como la discrepancia algebraica directa entre el valor objetivo y la respuesta emitida por la neurona: $e_k = y_{d,k} - y_k$

En la representación biopolar simétrica con función `hardlims`, surgen exclusivamente tres escenarios discretos posibles para e_k :

Salida Deseada (y_d)	Salida Obtenida (y)	Error ($e = y_d - y$)	Interpretación y Acción del Algoritmo
+1	+1	0	Clasificación Correcta: No se alteran los pesos ni el sesgo ($\Delta W = 0, \Delta b = 0$)
-1	-1	0	Clasificación Correcta: No se alteran los pesos ni el sesgo ($\Delta W = 0, \Delta b = 0$)
+1	-1	+2	Falso Negativo (falta de excitación): La neurona debió disparar pero no superó el umbral ($a < 0$). Se aplica: $W^{nuevo} = W^{actual} + 2X_k$ $b^{nuevo} = b^{actual} + 2$ <i>Efecto geométrico:</i> Rota el vector de pesos sumándole el vector X_k , acercando el hiperplano hacia la región patrón.
-1	+1	-2	Falso Positivo (exceso de excitación): La neurona disparó indebidamente (). Se aplica: $W^{nuevo} = W^{actual} - 2X_k$ $b^{nuevo} = b^{actual} - 2$ <i>Efecto geométrico:</i> Rota el vector de pesos en sentido contrario al vector X_k , alejando el hiperplano de la región infractora.

Criterio de Parada y Convergencia:

El proceso se itera secuencialmente a través de épocas completas (recorridos exhaustivos de los N patrones de entrenamiento). El algoritmo finaliza con éxito en el instante en que el error global acumulado de la época alcanza el valor cero: $E_{global} = \sum_{k=1}^N |e_k| = 0$. Por el **Teorema de Convergencia del Perceptrón**, si el problema linealmente separable en el espacio de características, el algoritmo garantiza converger a un conjunto finito de pesos W^* y sesgo b^* en un número acotado de épocas.

DESARROLLO DE LA ACTIVIDAD – CASO DE ESTUDIO 1: CONDICIONAMIENTO DE PALOMA:

- **Planteamiento del Problema y Variables de Entrada/Salida.**

Contexto del Experimento

El presente caso de estudio aborda la simulación de un experimento clásico de **condicionamiento operante o instrumental** en una paloma dentro de una cámara experimental (*caja de Skinner*). El dispositivo cuenta con dos pulsadores luminosos adyacentes (uno izquierdo y uno derecho) y un dispensador automático de comida.

La política experimental impone las siguientes condiciones:

1. **Con luces encendidas:** Si al menos uno de los dos pulsadores se encuentra encendido, la paloma recibe alimento picando indistintamente en cualquiera de los pulsadores.
2. **Con luces apagados (ambos pulsadores apagados):** La paloma debe discriminar el entorno y picar estrictamente el **pulsador derecho** para conseguir la recompensa. Si bajo esta condición pica el pulsador izquierdo, no recibe alimento (fracaso).

El objetivo consiste en entrenar un Perceptrón Simple para que aprenda a discriminar estos estímulos ambientales y emule la toma de decisiones óptima ante cada combinación posible.

Las tres entradas y el valor deseado operan estrictamente en el dominio bipolar $\{-1, +1\}$.

- **Codificación de entradas:**
 - x_1 : Pulsador Izquierdo $\rightarrow +1$ si encendido; -1 si apagado
 - x_2 : Pulsador Derecho $\rightarrow +1$ si encendido; -1 si apagado
 - x_3 : Acción de la Paloma $\rightarrow +1$ si pica el izquierdo; -1 si pica el derecho
- **Salida Deseada (y_d):** $+1 = \text{Éxito (comida)}$; $-1 = \text{Fracaso (sin comida)}$.
- **Función de Activación:** Función escalón bipolar **hardlims(a)**:

$$f(a) = \text{hardlims}(a) = \begin{cases} +1 & \text{si } a \geq 0 \\ -1 & \text{si } a < 0 \end{cases}$$

- **Tabla de Verdad Bipolar (8 Patrones).**

Justificación y Deducción de los Patrones

Dado que el sistema cuenta con 3 variables independientes de entrada (x_1, x_2, x_3) y cada una adopta dos valores posibles (± 1), existen $2^3 = 8$ combinaciones ambientales de comportamiento:

- En los patrones $P_3, P_4, P_5, P_6, P_7, P_8$, al menos uno de los pulsadores está encendido ($x_1 = +1$ o $x_2 = +1$), por lo que cualquier acción de la paloma conduce a la recompensa ($y_d = +1$).
- Cuando ambos pulsadores están apagados ($x_1 = -1, x_2 = -1$), si la paloma pica a la derecha ($x_3 = -1$, patrón P_1) obtiene comida ($y_d = +1$).
- El único escenario penalizado es el patrón P_2 , donde ambos pulsadores están apagados y la paloma pica a la izquierda ($x_3 = +1$), resultando en fracaso ($y_d = -1$).

Patrón	x_1	x_2	x_3	Condición Ambiental	Acción Tomada	y_d
P_1	-1	-1	-1	Ambos Apagados	Pica Pulsador Derecho	+1
P_2	-1	-1	+1	Ambos Apagados	Pica Pulsador Izquierdo	-1
P_3	-1	+1	-1	Derecho Encendido	Pica Pulsador Derecho	+1
P_4	-1	+1	+1	Derecho Encendido	Pica Pulsador Izquierdo	+1
P_5	+1	-1	-1	Izquierdo Encendido	Pica Pulsador Derecho	+1
P_6	+1	-1	+1	Izquierdo Encendido	Pica Pulsador Izquierdo	+1
P_7	+1	+1	-1	Ambos Encendidos	Pica Pulsador Derecho	+1
P_8	+1	+1	+1	Ambos Encendidos	Pica Pulsador Izquierdo	+1

- **Condiciones Iniciales (pesos W^0 y sesgo b^0).**

Origen y Justificación de los Valores

De acuerdo con la metodología algorítmica vista en clase (*Pseudocódigo del Perceptrón simple*), el modelo debe iniciar en un estado de **tabula rasa** (sin conocimiento previo). Por consiguiente:

1. Los **pesos sinápticos iniciales** se generan de forma pseudoaleatoria con magnitudes pequeñas en el intervalo simétrico $[-1, 1]$, garantizando que la red comience explorando el espacio sin simetrías rígidas:

$$W^0_1, W^0_2, W^0_3 \in [-1, 1]$$

2. El **sesgo inicial (bias)** se establece aleatoriamente dentro del rango positivo $[0, 1]$, representando nivel basal de activación:

$$b^0 \in [0, 1]$$

Aplicando esta condición de inicialización aleatoria, se definen los parámetros de partida del sistema:

$$W^0 = [0.3, -0.9, -0.4]^T \quad b^0 = 0.2$$

- $W_1^0 = 0.3$ (peso asociado al pulsador izquierdo)
- $W_2^0 = -0.9$ (peso asociado al pulsador derecho)
- $W_3^0 = -0.4$ (peso asociado a la acción de picar de la paloma)
- $b^0 = 0.2$ (sesgo o umbral inicial)

Con estas condiciones arbitrarias, el Perceptrón comete errores sistemáticos que activarán la corrección por regla delta.

- **Curva de Aprendizaje (Evolución del Error Época tras Época).**

Formulación de la Regla de Aprendizaje Supervisado

El proceso de optimización del Perceptrón Simple opera bajo el paradigma de **aprendizaje supervisado**, en el cual la red se confronta secuencialmente a los pares de entrenamiento ($X_k, y_{d,k}$) para evaluar la discrepancia entre la respuesta calculada y la deseada.

Las ecuaciones matemáticas que gobiernan este proceso son:

1. Combinación lineal (Propagación hacia adelante):

$$a_k = W^T X_k + b = \sum_{j=1}^3 W_j x_{j,k} + b = W_1 x_{1,k} + W_2 x_{2,k} + W_3 x_{3,k} + b$$

2. Evaluación de Activación Bipolar:

$$y_k = \text{hardlims}(a_k) = \begin{cases} +1 & \text{si } a_k \geq 0 \\ -1 & \text{si } a_k < 0 \end{cases}$$

3. Cálculo del Error Individual: $e_k = y_{d,k} - y_k \in \{-2, 0, +2\}$

4. Ecuaciones de Actualización Sináptica (Regla Delta):

- Si $e_k = 0$: No existe error de clasificación; los pesos y el sesgo permanecen inalterados.
- Si $e_k \neq 0$: Se ajusta el vector de pesos y el sesgo en proporción directa al error y al vector de entrada:

$$W_j^{\text{nuevo}} = W_j^{\text{actual}} + e_k \cdot x_{j,k} \quad (\text{para } j = 1, 2, 3)$$

$$b^{\text{nuevo}} = b^{\text{actual}} + e_k$$

5. En cada época (recorrido exhaustivo de los $N = 8$ patrones), se calcula la suamtoria acumulada de errores absolutos:

$$E_{\text{global}} = \sum_{k=1}^8 |e_k|$$

El criterio de parada se alcanza cuando $E_{\text{global}} = 0$, garantizando que la red no comete ninguna equivocación sobre el conjunto de entrenamiento

Algoritmo de Entrenamiento del Perceptrón Simple (PS Bipolar)

El procedimiento iterativo que implementa formalmente la dinámica de aprendizaje descrita se estructura a continuación mediante pseudocódigo algorítmico documentado:

```
=====
ALGORITMO DE ENTRENAMIENTO DEL PERCEPTRÓN SIMPLE (PS) - REPRESENTACIÓN BIPOLAR
=====

ENTRADA:

- Conjunto de patrones de entrenamiento:  $\{(X_k, y_{d,k})\}$  para  $k = 1, 2, \dots, N$  ( $N = 8$ )
  donde  $X_k = [x_1, x_2, x_3]^T$  en  $\{-1, +1\}^3$  y  $y_{d,k}$  en  $\{-1, +1\}$ .

- Tolerancia de error permitida:  $E_{perm} = 0$ .

- Límite máximo de épocas de seguridad:  $Epocas\_Max = 100$ .

INICIALIZACIÓN:

-  $W = [W_1, W_2, W_3]^T$  // Pesos iniciales generados aleatoriamente en  $[-1, 1]$ 
-  $b$  // Sesgo inicial generado aleatoriamente en  $[0, 1]$ 
-  $Epoca = 0$  // Contador de épocas transcurridas
-  $Error\_Global = 1$  // Inicialización en valor no nulo para entrar al bucle

PROCESO ITERATIVO:

MIENTRAS ( $Error\_Global > E_{perm}$  Y  $Epoca < Epocas\_Max$ ) HACER:

     $Epoca = Epoca + 1$ 

     $Error\_Global = 0$  // Reinicio del acumulador de error para la nueva época

    PARA CADA patrón  $k$  DESDE 1 HASTA  $N$  HACER:

        // 1. Cálculo de la suma neta ponderada (Combinación lineal)
         $a = (W_1 * x_{1,k}) + (W_2 * x_{2,k}) + (W_3 * x_{3,k}) + b$ 

        // 2. Aplicación de la función de activación escalón bipolar (hardlims)
        SI  $a \geq 0$  ENTONCES:
             $y = +1$ 
        SINO:
             $y = -1$ 
        FIN SI

        // 3. Determinación de la señal de error individual
         $e_k = y_{d,k} - y$ 
```

```

// 4. Actualización adaptativa de parámetros mediante la Regla Delta

SI e_k != 0 ENTONCES:

    // Actualización de cada peso sináptico

    W_1 = W_1 + (e_k * x_1,k)

    W_2 = W_2 + (e_k * x_2,k)

    W_3 = W_3 + (e_k * x_3,k)

    // Actualización del término independiente de sesgo (bias)

    b = b + e_k

    // Acumulación de la magnitud del error en la época

    Error_Global = Error_Global + ABS(e_k)

FIN SI

FIN PARA

FIN MIENTRAS

SALIDA:

- Vector de pesos calibrados:  $W^* = [W_1, W_2, W_3]^T$ 

- Sesgo calibrado:  $b^*$ 

- Total de épocas requeridas hasta la convergencia

=====

```

Implementación Computacional en Python (Caso 1: Paloma)

En estricta correspondencia con el pseudocódigo formal y la arquitectura modular del código final desarrollado en el proyecto (**src/perceptron.py** y **src/consola/caso_1_paloma.py**), a continuación se presenta el script en Python que implementa la clase **PerceptronSimpleBipolar** y ejecuta el entrenamiento supervisado mediante la Regla Delta para este caso de estudio:

```

# -*- coding: utf-8 -*-

"""

Caso de Estudio 1: Simulación del Condicionamiento Instrumental de la Paloma

Implementación algorítmica del Perceptron Simple Bipolar con función hardlims.

"""

import numpy as np

```

```

class PerceptronSimpleBipolar:

    """

    Red Neuronal Monocapa tipo Perceptron Simple Bipolar.

    Atributos:

        n_entradas (int): Numero de entradas de la red.

        W (np.ndarray): Vector de pesos sinapticos.

        b (float): Termino de sesgo (bias).

        historial_error (list): Registro del error global por epoca.

        epocas_entrenadas (int): Total de epocas ejecutadas.

    """

    def __init__(self, n_entradas=3, pesos_iniciales=None, sesgo_inicial=None):

        """Inicializa pesos y sesgo de la neurona."""

        self.n_entradas = n_entradas

        self.W = np.array(pesos_iniciales, dtype=float) if pesos_iniciales is not None else
np.round(np.random.uniform(-1.0, 1.0, size=n_entradas), 2)

        self.b = float(sesgo_inicial) if sesgo_inicial is not None else
float(np.round(np.random.uniform(0.0, 1.0), 2))

        self.historial_error = []

        self.epocas_entrenadas = 0

    @staticmethod
    def hardlims(a):

        """Funcion de activacion escalon simetrica hardlims(a)."""

        return 1 if a >= 0 else -1

    def propagacion(self, X):

        """Calcula la combinacion lineal  $a = W^T * X + b$ ."""

        return float(np.dot(self.W, X) + self.b)

    def predecir(self, X):

        """Clasifica una entrada mediante hardlims(a)."""

        return self.hardlims(self.propagacion(X))

    def entrenar(self, X_train, y_train, epocas_max=100, tolerancia=0, verbose=True):

```

```

"""
Ejecuta el ciclo de entrenamiento supervisado con la Regla Delta.
"""

N = len(y_train)

self.historial_error = []

self.epocas_entrenadas = 0

while self.epocas_entrenadas < epocas_max:

    self.epocas_entrenadas += 1

    error_global = 0

    for k in range(N):

        X_k = np.array(X_train[k], dtype=float)

        yd_k = int(y_train[k])

        # Propagacion y evaluacion

        a = self.propagacion(X_k)

        y = self.hardlims(a)

        e = yd_k - y

        # Actualizacion sinaptica por Regla Delta

        if e != 0:

            delta_W = e * X_k

            self.W += delta_W

            self.b += e

            error_global += abs(e)

        self.historial_error.append(error_global)

    # Criterio de parada

    if error_global <= tolerancia:

        break

    return self.historial_error

def resolver_caso_1():

```

```

"""Ejecuta el entrenamiento y verificación para el Caso 1 (Paloma)."""

# 1. Matriz de patrones bipolares de entrenamiento
# X = [x1 (Pulsador Izq), x2 (Pulsador Der), x3 (Acción Paloma)]

X = np.array([
    [-1, -1, -1],
    [-1, -1,  1],
    [-1,  1, -1],
    [-1,  1,  1],
    [ 1, -1, -1],
    [ 1, -1,  1],
    [ 1,  1, -1],
    [ 1,  1,  1]
], dtype=int)

yd = np.array([1, -1, 1, 1, 1, 1, 1, 1], dtype=int)

# 2. Condiciones iniciales

w0 = [0.3, -0.9, -0.4]

b0 = 0.2

# 3. Instanciar y entrenar la red

ps = PerceptronSimpleBipolar(n_entradas=3, pesos_iniciales=w0, sesgo_inicial=b0)

historial_error = ps.entrenar(X, yd, epocas_max=10, tolerancia=0, verbose=False)

# 4. Verificación de clasificación con parámetros calibrados

print(f"Convergencia alcanzada en Epoca {ps.epocas_entrenadas} con Error Global = {historial_error[-1]}")

print(f"Pesos finales calibrados W* = {list(ps.W)}, Sesgo final b* = {ps.b}")

for k in range(len(yd)):
    a = ps.propagacion(X[k])
    y = ps.predecir(X[k])

    print(f"P{k+1}: X={list(X[k])}, yd={yd[k]}, y={y}, a={a:.1f} -> {'CORRECTO' if y == yd[k] else 'ERROR'}")

if __name__ == "__main__":
    resolver_caso_1()

```

DESARROLLO ARITMÉTICO: EPOCA 1

- **Estado inicial:** $W = [0.3, -0.9, -0.4]^T, b = 0.2. E_{global} = 0$
- **Iteración 1 ($P_1: X = [-1, -1, -1]^T, y_d = +1$):**

$$a = (0.3)(-1) + (-0.9)(-1) + (-0.4)(-1) + (0.2) = -0.3 + 0.9 + 0.4 + 0.2 = 1.2$$

Como $a = 1.2 \geq 0 \rightarrow y = +1$

$$e = (+1) - (+1) = 0 \rightarrow \text{Acierto. Sin cambios. } E_{global} = 0$$

- **Iteración 2 ($P_2: X = [-1, -1, +1]^T, y_d = -1$):**

$$a = (0.3)(-1) + (-0.9)(-1) + (-0.4)(+1) + (0.2) = -0.3 + 0.9 - 0.4 + 0.2 = 0.4$$

Como $a = 0.4 \geq 0 \rightarrow y = +1$

$$e = (-1) - (+1) = -2 \neq 0 \rightarrow \text{Falso positivo. Correccion:}$$

$$W^{nuevo}_1 = 0.3 + (-2)(-1) = 0.3 + 2.0 = 2.3$$

$$W^{nuevo}_2 = -0.9 + (-2)(-1) = -0.9 + 2.0 = 1.1$$

$$(W^{nuevo})_3 = -0.4 + (-2)(-1) = -0.4 - 2.0 = -2.4$$

$$b^{nuevo} = 0.2 + (-2) = -1.8$$

$$W = [2.3, 1.1, -2.4]^T \quad b = -1.8 \quad E_{global} = 0 + |-2| = 2$$

- **Iteración 3 ($P_3: X = [-1, +1, -1]^T, y_d = +1$):**

$$a = (2.3)(-1) + (1.1)(+1) + (-2.4)(-1) + (-1.8) = -2.3 + 1.1 + 2.4 - 1.8 = -0.6$$

Como $a = -0.6 < 0 \rightarrow y = -1$

$$e = (+1) - (-1) = +2 \neq 0 \rightarrow \text{Falso negativo Correccion:}$$

$$W^{nuevo}_1 = 2.3 + (+2)(-1) = 2.3 - 2.0 = 0.3$$

$$(W^{nuevo})_2 = 1.1 + (+2)(+1) = 1.1 + 2.0 = 3.1$$

$$(W^{nuevo})_3 = -2.4 + (+2)(-1) = -2.4 - 2.0 = -4.4$$

$$b^{nuevo} = -1.8 + (+2) = 0.2$$

$$W = [0.3, 3.1, -4.4]^T \quad b = 0.2 \quad E_{global} = 2 + |+2| = 4$$

- **Iteración 4 ($P_4: X = [-1, +1, +1]^T, y_d = +1$):**

$$a = (0.3)(-1) + (3.1)(+1) + (-4.4)(+1) + (0.2) = -0.3 + 3.1 - 4.4 + 0.2 = -1.4$$

Como $a = -1.4 < 0 \rightarrow y = -1$

$$e = (+1) - (-1) = +2 \neq 0 \rightarrow \text{Falso negativo Correccion:}$$

$$W^{nuevo}_1(+2)(+2) = 0.3 + (-1) = 0.3 - 2.0 = 1.7$$

$$(W^{nuevo})_2 = 3.1 + (+2)(+1) = 3.1 + 2.0 = 5.1$$

$$(W^{nuevo})_3 = -4.4 + (+2)(+1) = -4.4 + 2.0 = -2.4$$

$$b^{nuevo} = 0.2 + (+2) = 2.2$$

$$W = [-1.7, 5.1, -2.4]^T \quad b = 2.2 \quad E_{global} = 4 + | +2 | = 6$$

- **Iteración 5 ($P_5: X = [+1, -1, -1]^T, y_d = +1$):**

$$a = (-1.7)(+1) + (5.1)(-1) + (-2.4)(-1) + (2.2) = -1.7 - 5.1 + 2.4 + 2.2 = -2.2$$

Como $a = -2.2 < 0 \rightarrow y = -1$

$$e = (+1) - (-1) = +2 \neq 0 \rightarrow \text{Falso negativo Correccion:}$$

$$(W^{nuevo})_1 = -1.7 + (+2)(+1) = -1.7 + 2.0 = 0.3$$

$$(W^{nuevo})_2 = 5.1 + (+2)(-1) = 5.1 - 2.0 = 3.1$$

$$(W^{nuevo})_3 = -2.4 + (+2)(-1) = -2.4 - 2.0 = -4.4$$

$$b^{nuevo} = 2.2 + (+2) = 4.2$$

$$W = [0.3, 3.1, -4.4]^T \quad b = 4.2 \quad E_{global} = 6 + | +2 | = 8$$

- **Iteración 6 ($P_6: X = [+1, -1, +1]^T, y_d = +1$):**

$$(0.3)(+1)a = + (3.1)(-1) + (-4.4)(-1) + (4.2) = 0.3 - 3.1 - 4.4 + 4.2 = -3.0$$

Como $a = -3.0 < 0 \rightarrow y = -1$

$$e = (+1) - (-1) = +2 \neq 0 \rightarrow \text{Falso negativo Correccion:}$$

$$(W^{nuevo})_1 = 0.3 + (+2)(+1) = 0.3 + 2.0 = 2.3$$

$$(W^{nuevo})_2 = 3.1 + (+2)(-1) = 3.1 - 2.0 = 1.1$$

$$(W^{nuevo})_3 = -4.4 + (+2)(+1) = -4.4 + 2.0 = -2.4$$

$$b^{nuevo} = 4.2 + (+2) = 6.2$$

$$W = [2.3, 1.1, -2.4]^T \quad b = 6.2 \quad E_{global} = 8 + | +2 | = 10$$

- **Iteración 7 ($P_7: X = [+1, +1, -1]^T, y_d = +1$):**

$$a = (2.3)(+1) + (1.1)(+1) + (-2.4)(-1) + (6.2) = 2.3 + 1.1 + 2.4 + 6.2 = 12.0 \geq 0 \rightarrow y = +1$$

$$e = (+1) - (+1) = 0 \rightarrow \text{Sin cambios. } E_{global} = 10$$

- **Iteración 8 ($P_8: X = [+1, +1, +1]^T, y_d = +1$):**

$$a = (2.3)(+1) + (1.1)(+1) + (-2.4)(+1) + (6.2) = 2.3 + 1.1 - 2.4 + 6.2 = 7.2 \geq 0 \rightarrow y = +1$$

$$e = (+1) - (+1) = 0 \rightarrow \text{Sin cambios. } E_{global} = 10$$

FIN DE ÉPOCA 1: $E_{global} = 10 \neq 0$. Hubo 5 fallos. Se inicia la Epoca 2 con:

$$W = [2.3, 1.1, -2.4]^T \text{ y } b = 6.2$$

DESARROLLO ARITMÉTICO: EPOCA 2

- **Estado inicial:** $W = [2.3, 1.1, -2.4]^T, b = 6.2. E_{global} = 0$.
- **Iteración 1 (P_1):** $a = (2.3)(-1) + (1.1)(-1) + (-2.4)(-1) + 6.2 = 5.2 \geq 0 \rightarrow y = +1$

$$e = 0 \rightarrow E_{global} = 0$$

- **Iteración 2 (P_2):** $a = (2.3)(-1) + (1.1)(-1) + (-2.4)(+1) + 6.2 = 0.4 \geq 0 \rightarrow y = +1$

$$e = (-1) - (+1) = -2 \neq 0 \rightarrow \text{Actualizacion:}$$

$$W_1 = 2.3 + (-2)(-1) = 4.3$$

$$W_2 = 1.1 + (-2)(-1) = 3.1$$

$$W_3 = -2.4 + (-2)(+1) = -4.4$$

$$b = 6.2 + (-2) = 4.2$$

$$W = [4.3, 3.1, -4.4]^T, b = 4.2. E_{global} = 2$$

- **Iteración 3 (P_3):** $a = (4.3)(-1) + (3.1)(+1) + (-4.4)(-1) + 4.2 = 7.4 \geq 0 \rightarrow y = +1$

$$e = 0 \rightarrow E_{global} = 2$$

- **Iteración 4 (P_4):** $a = (4.3)(-1) + (3.1)(+1) + (-4.4)(+1) + 4.2 = -1.4 < 0 \rightarrow y = -1$

$$e = (+1) - (-1) = +2 \neq 0 \rightarrow \text{Actualizacion:}$$

$$W_1 = 4.3 + 2(-1) = 2.3$$

$$W_2 = 3.1 + 2(1) = 5.1$$

$$W_3 = -4.4 + 2(1) = -2.4$$

$$b = 4.2 + 2 = 6.2$$

$$W = [2.3, 5.1, -2.4]^T, b = 6.2. E_{global} = 4$$

- **Iteración 5 (P_5):**

$$a = (2.3)(1) + (5.1)(-1) + (-2.4)(-1) + 6.2 = 5.8 \geq 0 \rightarrow y = +1$$

$$e = 0 \rightarrow E_{global} = 4$$

- **Iteración 6 (P_6):** $a = (2.3)(1) + (5.1)(-1) + (-2.4)(1) + 6.2 = 1.0 \geq 0 \rightarrow y = +1$

$$e = 0 \rightarrow E_{global} = 4$$

- **Iteración 7 (P_7):** $a = (2.3)(1) + (5.1)(1) + (-2.4)(-1) + 6.2 = 2.3 \geq 0 \rightarrow y = +1$

$$e = 0 \rightarrow E_{global} = 4$$

- **Iteración 8 (P_8):** $a = (2.3)(1) + (5.1)(1) + (-2.4)(1) + 6.2 = 11.2 \geq 0 \rightarrow y = +1$

$$e = 0 \rightarrow E_{global} = 4$$

FIN DE ÉPOCA 2: $E_{global} = 4 \neq 0$. El error se redujo de 10 a 4 (solo 2 fallos). Se inicia la Epoca 3 con:

$$W = [2.3, 5.1, -2.4]^T \text{ y } b = 6.2$$

DESARROLLO ARITMÉTICO: EPOCA 3

- **Estado inicial:** $W = [2.3, 5.1, -2.4]^T, b = 6.2, E_{global} = 0$
- **Iteración 1 (P_1):** $a = (2.3)(-1) + (5.1)(-1) + (-2.4)(-1) + 6.2 = 1.2 \geq 0 \rightarrow y = +1$

$$e = 0 \rightarrow E_{global} = 0$$

- **Iteración 2 (P_2):** $a = (2.3)(-1) + (5.1)(-1) + (-2.4)(1) + 6.2 = -3.6 < 0 \rightarrow y = -1$

$$e = 0 \rightarrow E_{global} = 0$$

- **Iteración 3 (P_3):** $a = (2.3)(-1) + (5.1)(1) + (-2.4)(-1) + 6.2 = 11.4 \geq 0 \rightarrow y = +1$

$$e = 0 \rightarrow E_{global} = 0$$

- **Iteración 4 (P_4):** $a = (2.3)(-1) + (5.1)(1) + (-2.4)(1) + 6.2 = 6.6 \geq 0 \rightarrow y = +1$

$$e = 0 \rightarrow E_{global} = 0$$

- **Iteración 5 (P_5):** $a = (2.3)(1) + (5.1)(-1) + (-2.4)(-1) + 6.2 = 5.8 \geq 0 \rightarrow y = +1$

$$e = 0 \rightarrow E_{global} = 0$$

- **Iteración 6 (P_6):** $a = (2.3)(1) + (5.1)(-1) + (-2.4)(1) + 6.2 = 1.0 \geq 0 \rightarrow y = +1$

$$e = 0 \rightarrow E_{global} = 0$$

- **Iteración 7 (P_7):** $a = (2.3)(1) + (5.1)(1) + (-2.4)(-1) + 6.2 = 16.0 \geq 0 \rightarrow y = +1$

$$e = 0 \rightarrow E_{global} = 0$$

- **Iteración 8 (P_8):** $a = (2.3)(1) + (5.1)(1) + (-2.4)(1) + 6.2 = 11.2 \geq 0 \rightarrow y = +1, e = 0 \rightarrow E_{global} = 0$

FIN DE ÉPOCA 3: $E_{global} = 0$. **CONVERGENCIA ALCANZADA** El Percetrón clasifica sin errores los 8 patrones.

Registro Numérico de la Curva de Aprendizaje

Época	Patrones con Fallo	$E_{global} = \sum$	e_k
Época 1	5/8	10	En entrenamiento
Época 2	2/8	4	En entrenamiento
Época 3	0/8	0	CONVERGENCIA ÓPTIMA

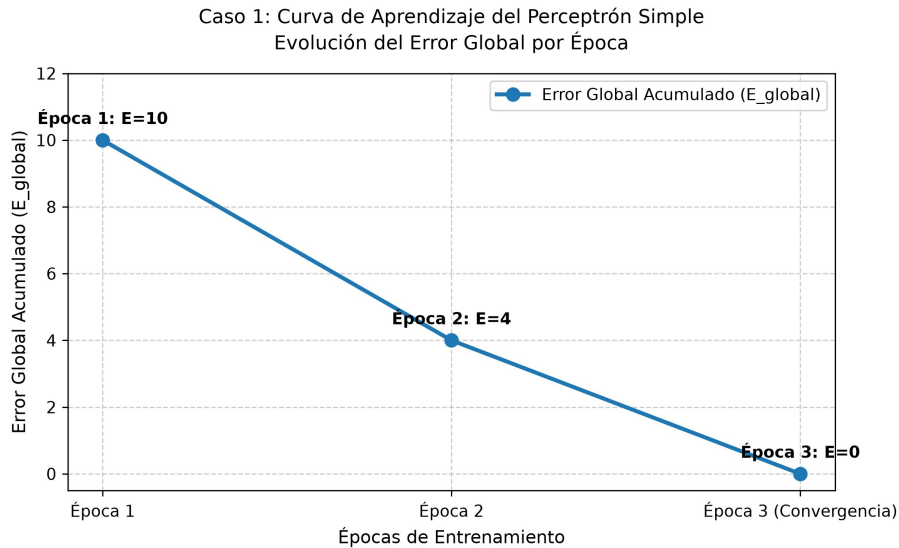


Figura 1. Curva de aprendizaje: Evolución del error global acumulado (E_{global}) a lo largo de las épocas de entrenamiento en el caso 1 (condicionamiento de la Paloma) hasta alcanzar la convergencia completa en cero errores ($E_{global} = 0$) en la Época 3.

- **Parámetros Calibrados Finales (W^* y b^*).**

Los parámetros óptimos finales resultantes de la convergencia son:

$$W^* = \begin{bmatrix} w_1^* \\ w_2^* \\ w_3^* \end{bmatrix} = \begin{bmatrix} 2.3 \\ 5.1 \\ -2.4 \end{bmatrix} \quad b^* = 6.2$$

Interpretación Física de los Parámetros Aprendidos:

1. $w_2^* = +5.1$ (**Peso Dominante**): El pulsador derecho encendido es el estímulo de mayor relevancia sináptica para obtener el alimento.
2. $w_1^* = +2.3$ (**Peso Dominante**): El pulsador izquierdo encendido ejerce un efecto excitatorio complementario positivo.
3. $w_3^* = -2.4$ (**Peso Inhibitorio a la Izquierda / Excitatorio a la Derecha**): Al estar codificada la acción derecha como $x_3 = -1$, el producto $(-2.4)(-1) = +2.4$ genera la excitación hacia la respuesta correcta, mientras que picar a la izquierda ($x_3 = +1$) resta excitación.
4. $b^* = +6.2$ (**Sesgo Alto**): Dado que 7 de los 8 patrones corresponden a éxito ($y_d = +1$), el sistema aprendió un sesgo positivo alto que sitúa el estado basal de la neurona en activación por defecto, requiriendo una inhibición conjunta específica para cambiar de decisión.

Tabla de Verificación de Salida con los Parámetros Finales

Patrón	Combinación Lineal $a = W^{*T}X + b^*$	Valor de a	$y = \text{hardlims}(a)$	y_d	Estado
P_1	$(2.3)(-1) + (5.1)(-1) + (-2.4)(-1) + 6.2 = -2.3 - 5.1 + 2.4 + 6.2$	+ 1.2	+ 1	+ 1	Correcto
P_2	$(2.3)(-1) + (5.1)(-1) + (-2.4)(+1) + 6.2 = -2.3 - 5.1 - 2.4 + 6.2$	- 3.6	- 1	- 1	Correcto
P_3	$(2.3)(-1) + (5.1)(+1) + (-2.4)(-1) + 6.2 = -2.3 + 5.1 + 2.4 + 6.2$	+ 11.4	+ 1	+ 1	Correcto
P_4	$(2.3)(-1) + (5.1)(+1) + (-2.4)(+1) + 6.2 = -2.3 + 5.1 - 2.4 + 6.2$	+ 6.6	+ 1	+ 1	Correcto
P_5	$(2.3)(+1) + (5.1)(-1) + (-2.4)(-1) + 6.2 = 2.3 - 5.1 + 2.4 + 6.2$	+ 5.8	+ 1	+ 1	Correcto
P_6	$(2.3)(+1) + (5.1)(-1) + (-2.4)(+1) + 6.2 = 2.3 - 5.1 - 2.4 + 6.2$	+ 1.0	+ 1	+ 1	Correcto
P_7	$(2.3)(+1) + (5.1)(+1) + (-2.4)(-1) + 6.2 = 2.3 + 5.1 + 2.4 + 6.2$	+ 16.0	+ 1	+ 1	Correcto
P_8	$(2.3)(+1) + (5.1)(+1) + (-2.4)(+1) + 6.2 = 2.3 + 5.1 - 2.4 + 6.2$	+ 11.2	+ 1	+ 1	Correcto

Efectividad del modelo: 8/8 patrones clasificados con exactitud (100%)

- **Ecuación Analítica del Hiperplano y Gráfica 3D de los Patrones.**

Dedución de la Ecuación del Hiperplano

La frontera geométrica de decisión se ubica donde la salida neta es nula ($a = 0$):

$$(W^*)_1 x_1 + (W^*)_2 x_2 + (W^*)_3 x_3 + b^* = 0$$

Sustituyendo los parámetros calibrados:

$$2.3x_1 + 5.1x_2 - 2.4x_3 + 6.2 = 0$$

Despejando x_3 para formular la superficie en función de (x_1, x_2) :

$$2.4x_3 = 2.3x_1 + 5.1x_2 + 6.2 \rightarrow x_3 = \frac{2.3}{2.4}x_1 + \frac{5.1}{2.4}x_2 + \frac{6.2}{2.4}$$

$$x_3 \approx 0.9583x_1 + 2.1250x_2 + 2.5833$$

Interpretación Geométrica Espacial:

- **Vector normal:** $W^* = [2.3, 5.1, -2.4]^T$, ortogonal al hiperplano separador.
- **Separación Lineal:** El único patrón con salida negativa es $P_2 = (-1, -1, +1)$, el cual produce $a = -3.6 < 0$ y se sitúa en el semiespacio opuesto al resto de los 7 patrones ($a > 0$). Esto verifica que el problema es **linealmente separable en R^3** .

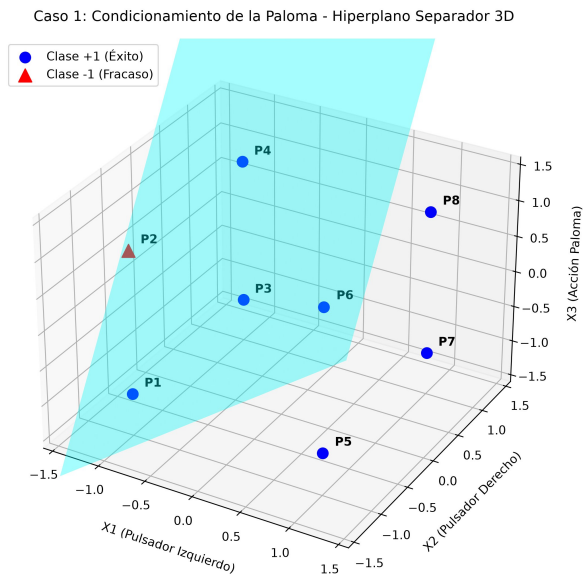


Figura 2. Representación geométrica tridimensional del hiperplano de decisión ($2.3x_1 + 5.1x_2 - 2.4x_3 + 6.2 = 0$) y la distribución de los 8 patrones en el espacio bipolar $\{-1, +1\}^3$. Se observa cómo el hiperplano aísla en el semiespacio inferior al patrón de fracaso P_2 (marcador rojo) de los 7 patrones de éxito (marcadores azules)

DESARROLLO DE LA ACTIVIDAD – CASO DE ESTUDIO 2: DIAGNÓSTICO MÉDICO POR SÍNTOMAS:

- **Planteamiento del Problema y Criterio de Decisión Clínica.**

Contexto del Problema Clínico

En la práctica médica preliminar, los profesionales de la salud se enfrentan constantemente a la necesidad de evaluar sintomatologías concurrentes para discernir entre cuadros clínicos de evolución benigna o asintomática frente a estados patológicos confirmados que requieren intervención terapéutica inmediata.

El presente caso de estudio aborda el diseño, modelado y entrenamiento de un Perceptrón Simple como sistema de apoyo a la decisión diagnóstica (*Clinical Decision Support System*). El modelo evalúa la presencia o ausencia de tres síntomas patológicos fundamentales:

1. **Fiebre (x_1)**
2. **Dolor de Cabeza / Cefalea (x_2)**
3. **Fatiga / Astenia (x_3)**

Criterio de Decisión Clínica

El protocolo diagnóstico estipula una regla de clasificación basada en la acumulación de síntomas:

- **Paciente Enfermo ($y_d = +1$):** Si el paciente manifiesta **dos o más síntomas** (≥ 2), se diagnostica la presencia confirmada del cuadro patológico.
- **Paciente Sano ($y_d = -1$):** Si el paciente presenta **un solo síntoma o ninguno** (< 2), se clasifica como asintomático o caso leve que no califica para el diagnóstico positivo.

El objetivo consiste en lograr que la neurona artificial encuentre de forma autónoma el hiperplano que separa ambos estados de salud con un margen de error nulo.

Las tres variables de entrada y el diagnóstico resultante operan en el dominio estrictamente bipolar $\{-1, +1\}$.

- **Codificación de variables de entrada:**
 - x_1 (*Fiebre*): $\rightarrow +1$ Si presenta fiebre; -1 Si no presenta fiebre.
 - x_2 (*Dolor de Cabeza / Cefalea*): $\rightarrow -1$ Si presenta cefalea; $+1$ si no presenta dolor.
 - x_3 (*Fatiga/ Astenia*): $\rightarrow +1$ Si presenta fatiga; -1 si no presenta fatiga.
- **Salida Deseada (y_d):**
 - $y_d = +1$: Enfermo (diagnóstico positivo / patología confirmada).
 - $y_d = -1$: Sano (asintomático o cuadro leve).
 - **Función de Activación:** Función escalón bipolar **hardlims(a)**:

$$f(a) = \text{hardlims}(a) = \begin{cases} +1 & \text{si } a \geq 0 \\ -1 & \text{si } a < 0 \end{cases}$$

- **Tabla de Verdad Bipolar (8 Patrones de Salud/Enfermedad).**

Justificación y Deducción de los Patrones Clínicos

Dado que se evalúan 3 síntomas binarios independientes, surgen $2^3 = 8$ posibles perfiles sintomatológicos en la población de pacientes:

- **0 Síntomas activos:** Patrón $P_1 = [-1, -1, -1]$. Paciente asintomático → Sano ($y_d = -1$)
- **1 Síntoma activo:** Patrones P_2, P_3, P_5 . El paciente solo manifiesta uno de los tres síntomas → Cuadro leve / Sano ($y_d = -1$)
- **2 Síntomas activos:** Patrones P_4, P_6, P_7 . Concurrencia de dos síntomas → Patología confirmada / Enfermo ($y_d = +1$)
- **3 Síntomas activos:** Patrón $P_8 = [+1, +1, +1]^T$. Cuadro clínico severo con todos los síntomas → Enfermo ($y_d = +1$).

A diferencia del Caso 1 (donde existía una fuerte asimetría de 7 éxitos y 1 fracaso), el caso 2 presenta una distribución perfectamente balanceada: **4 patrones de clase negativa ($y_d = -1$)** y **4 patrones de clase positiva ($y_d = +1$)**.

Matriz de Patrones de Entrenamiento Clínico

Paciente	x_1 (Fiebre)	x_2 (Cefalea)	x_3 (Fatiga)	Conteo de Síntomas	Diagnóstico Clínico	Salida Deseada (y_d)
P_1	-1	-1	-1	0 síntomas	Asintomático (Sano)	-1
P_2	-1	-1	+1	1 síntoma	Leve (Sano)	-1
P_3	-1	+1	-1	1 síntoma	Leve (Sano)	-1
P_4	-1	+1	+1	2 síntomas	Patología confirmada	+1
P_5	+1	-1	-1	1 síntoma	Leve (Sano)	-1
P_6	+1	-1	+1	2 síntomas	Patología confirmada	+1
P_7	+1	+1	-1	2 síntomas	Patología confirmada	+1
P_8	+1	+1	+1	3 síntomas	Cuadro clínico severo	+1

- **Condiciones Iniciales (W^0 y b^0).**

En concordancia con los principios del aprendizaje artificial supervisado, la red parte sin información previa de la regla diagnóstica (*tabula rasa*):

1. Los **pesos iniciales** se generan de forma pseudoaleatoria dentro del intervalo simétrico $[-1, 1]$, evitando sesgos o simetrías prematuras en el espacio de características:

$$W_1^0, W_2^0, W_3^0 \in [-1, 1]$$

2. El **sesgo inicial (bias)** se selecciona aleatoriamente en el rango positivo $[0, 1]$, estableciendo un umbral inicial de disparo:

$$b^0 \in [0, 1]$$

Aplicando esta condición de partida aleatoria para el desarrollo principal del caso, se establecen los siguientes parámetros de inicialización:

$$W^0 = [-0.4, -0.7, 0.3]^T \quad b^0 = 0.1$$

- $W_1^0 = -0.4$ (*peso inicial asignado a la Fiebre*)
- $W_2^0 = -0.7$ (*peso inicial asignado a la Cefalea*)
- $W_3^0 = 0.3$ (*peso inicial asignado a la Fatiga*)
- $b^0 = -0.1$ (*sesgo inicial del clasificador*)

Bajo esta configuración inicial aleatoria, la red clasifica erróneamente varios perfiles de pacientes, lo que desencadena las correcciones sucesivas por regla delta.

- **Curva de Aprendizaje (Evolución del Error).**

Formulación de la Regla de Aprendizaje Supervisado

El proceso iterativo de calibración sináptica se rige por las siguientes ecuaciones fundamentales:

1. **Combinación Lineal (Propagación hacia adelante)**

$$a_k = W^T X_k + b = \sum_{j=1}^3 W_j x_{j,k} + b = W_1 x_{1,k} + W_2 x_{2,k} + W_3 x_{3,k} + b$$

2. **Evaluación de Activación Bipolar:**

$$y_k = \text{hardlims}(a_k) = \begin{cases} +1 & \text{si } a_k \geq 0 \\ -1 & \text{si } a_k < 0 \end{cases}$$

3. **Cálculo del Error Individual:**

$$e_k = y_{d,k} - y_k \in \{-2, 0, +2\}$$

4. **Ecuaciones de Actualización Sináptica (Regla Delta):**

- Si $e_k = 0$: Predicción correcta; los pesos y el sesgo no se modifican.
- Si $e_k \neq 0$: Se corrige la orientación del vector normal y la posición del hiperplano.

$$W_j^{\text{nuevo}} = W_j^{\text{actual}} + e_k \cdot x_{j,k} \quad (\text{para } j = 1, 2, 3)$$

$$b^{\text{nuevo}} = b^{\text{actual}} + e_k$$

5. **Criterio de Evaluación Global:** En cada época se acumula la magnitud de los errores observados:

$$E_{global} = \sum_{k=1}^8 |e_k|$$

El entrenamiento concluye exitosamente cuando $E_{global} = 0$

Algoritmo de Entrenamiento del Perceptrón Simple (PS Bipolar)

A continuación se detalla la estructura formal del algoritmo en pseudocódigo técnico comentado:

```
=====
ALGORITMO DE ENTRENAMIENTO DEL PERCEPTRÓN SIMPLE (PS) - DIAGNÓSTICO CLÍNICO BIPOLAR
=====

ENTRADA:

- Matriz de entrenamiento: {(X_k, y_d,k)} para k = 1, 2, ..., 8
  donde X_k = [Fiebre, Cefalea, Fatiga]^T en {-1, +1}^3 y y_d,k en {-1, +1}.
- Tolerancia de error: E_perm = 0.
- Épocas máximas de control: Epocas_Max = 100.

INICIALIZACIÓN:

- W = [-0.4, -0.7, 0.3]^T // Vector de pesos iniciales en [-1, 1]
- b = 0.1 // Sesgo inicial en [0, 1]
- Epoca = 0 // Contador de épocas
- Error_Global = 1 // Valor inicial para iniciar el bucle

PROCESO ITERATIVO:

MIENTRAS (Error_Global > E_perm Y Epoca < Epocas_Max) HACER:

    Epoca = Epoca + 1

    Error_Global = 0

    PARA CADA paciente k DESDE 1 HASTA 8 HACER:

        // 1. Salida neta: ponderación de síntomas con los pesos actuales
        a = (W_1 * x_1,k) + (W_2 * x_2,k) + (W_3 * x_3,k) + b

        // 2. Discriminación de clase mediante escalón bipolar
        SI a >= 0 ENTONCES:

            y = +1 // Diagnóstico preliminar: Enfermo
```

```

SINO:

    y = -1    // Diagnóstico preliminar: Sano

FIN SI

// 3. Comparación clínica y obtención del error individual
e_k = y_d,k - y

// 4. Adaptación sináptica por Regla Delta ante discrepancias
SI e_k != 0 ENTONCES:

    W_1 = W_1 + (e_k * x_1,k)

    W_2 = W_2 + (e_k * x_2,k)

    W_3 = W_3 + (e_k * x_3,k)

    b = b + e_k

    Error_Global = Error_Global + ABS(e_k)

FIN SI

FIN PARA

FIN MIENTRAS

SALIDA:

- Pesos finales calibrados: W* = [W_1, W_2, W_3]^T

- Sesgo final calibrado: b*

- Total de épocas requeridas

=====

```

Implementación Computacional en Python (Caso 2: Diagnóstico Médico)

En estricta correspondencia con el pseudocódigo formal y el diseño modular del código final desarrollado en el proyecto (`src/perceptron.py` y `src/consola/caso_2_diagnostico.py`), a continuación se presenta el script en Python que implementa la clase `PerceptronSimpleBipolar` y ejecuta tanto el experimento principal (convergencia en 3 épocas) como la experimentación comparativa de sensibilidad (convergencia acelerada en 2 épocas y 1 época):

```
# -*- coding: utf-8 -*-

"""

Caso de Estudio 2: Sistema de Diagnostico Medico Basado en Sintomas

Implementacion algoritmica del Perceptron Simple Bipolar con funcion
hardlims.

"""

import numpy as np

class PerceptronSimpleBipolar:

    """

    Red Neuronal Monocapa tipo Perceptron Simple Bipolar.

    Atributos:

        n_entradas (int): Numero de variables clinicas de entrada.

        W (np.ndarray): Vector de pesos sinapticos.

        b (float): Termino de sesgo (bias).

        historial_error (list): Registro del error global por epoca.

        epocas_entrenadas (int): Total de epocas ejecutadas.

    """

    def __init__(self, n_entradas=3, pesos_iniciales=None,
sesgo_inicial=None):

        """Inicializa pesos y sesgo de la neurona."""

        self.n_entradas = n_entradas

        self.W = np.array(pesos_iniciales, dtype=float) if pesos_iniciales is
not None else np.round(np.random.uniform(-1.0, 1.0, size=n_entradas), 2)

        self.b = float(sesgo_inicial) if sesgo_inicial is not None else
float(np.round(np.random.uniform(0.0, 1.0), 2))

        self.historial_error = []
```

```

        self.epocas_entrenadas = 0

    @staticmethod
    def hardlims(a):
        """Funcion de activacion escalon simetrica hardlims(a)."""
        return 1 if a >= 0 else -1

    def propagacion(self, X):
        """Calcula la combinacion lineal ponderada  $a = W^T * X + b$ ."""
        return float(np.dot(self.W, X) + self.b)

    def predecir(self, X):
        """Clasifica el patron clinico mediante hardlims(a)."""
        return self.hardlims(self.propagacion(X))

    def entrenar(self, X_train, y_train, epocas_max=100, tolerancia=0,
verbose=True):
        """
        Ejecuta el ciclo de entrenamiento supervisado con la Regla Delta.
        """
        N = len(y_train)
        self.historial_error = []
        self.epocas_entrenadas = 0

        while self.epocas_entrenadas < epocas_max:
            self.epocas_entrenadas += 1
            error_global = 0

            for k in range(N):
                X_k = np.array(X_train[k], dtype=float)
                yd_k = int(y_train[k])

                # Propagacion y clasificacion
                a = self.propagacion(X_k)
                y = self.hardlims(a)

```

```

        e = yd_k - y

        # Ajuste sinaptico adaptativo por Regla Delta
        if e != 0:

            delta_W = e * X_k

            self.W += delta_W

            self.b += e

            error_global += abs(e)

        self.historial_error.append(error_global)

        # Condicion de parada: error nulo
        if error_global <= tolerancia:

            break

    return self.historial_error

def resolver_caso_2():

    """Ejecuta el experimento principal y la comparativa de sensibilidad
    (Caso 2)."""

    # 1. Matriz de patrones clinicos bipolares

    # X = [x1 (Fiebre), x2 (Cefalea), x3 (Fatiga)]
    X = np.array([

        [-1, -1, -1],

        [-1, -1, 1],

        [-1, 1, -1],

        [-1, 1, 1],

        [ 1, -1, -1],

        [ 1, -1, 1],

        [ 1, 1, -1],

        [ 1, 1, 1]

    ], dtype=int)

    yd = np.array([-1, -1, -1, 1, -1, 1, 1, 1], dtype=int)

```

```

# 2. Experimento Principal (Convergencia en 3 Epocas)

W0 = [-0.4, -0.7, 0.3]

b0 = 0.1

print("--- 1. EXPERIMENTO PRINCIPAL (3 EPOCAS) ---")

ps_principal = PerceptronSimpleBipolar(n_entradas=3, pesos_iniciales=W0,
sesgo_inicial=b0)

hist_p = ps_principal.entrenar(X, yd, epocas_max=10, tolerancia=0,
verbose=False)

print(f"Convergencia en Epoca {ps_principal.epocas_entrenadas} |
Historial: {hist_p}")

print(f"Pesos finales calibrados W* = {list(ps_principal.W)}, Sesgo final
b* = {ps_principal.b}\n")


# 3. Configuracion Optimizada (Convergencia en 2 Epocas)

print("--- 2. CONFIGURACION OPTIMIZADA (2 EPOCAS) ---")

ps_2ep = PerceptronSimpleBipolar(n_entradas=3, pesos_iniciales=[-0.9, -
0.9, -0.9], sesgo_inicial=0.9)

hist_2ep = ps_2ep.entrenar(X, yd, epocas_max=10, tolerancia=0,
verbose=False)

print(f"Convergencia en Epoca {ps_2ep.epocas_entrenadas} | Historial:
{hist_2ep}")

print(f"Pesos finales calibrados W* = {list(ps_2ep.W)}, Sesgo final b* =
{ps_2ep.b}\n")


# 4. Solucion Directa (Convergencia en 1 Epoca)

print("--- 3. SOLUCION DIRECTA (1 EPOCA) ---")

ps_1ep = PerceptronSimpleBipolar(n_entradas=3, pesos_iniciales=[1.0, 1.0,
1.0], sesgo_inicial=0.0)

hist_1ep = ps_1ep.entrenar(X, yd, epocas_max=10, tolerancia=0,
verbose=False)

print(f"Convergencia en Epoca {ps_1ep.epocas_entrenadas} | Historial:
{hist_1ep}")

print(f"Pesos finales calibrados W* = {list(ps_1ep.W)}, Sesgo final b* =
{ps_1ep.b}")

if __name__ == "__main__":

    resolver_caso_2()

```

DESARROLLO ARITMÉTICO: EPOCA 1

- **Estado Inicial:** $W = [-0.4, -0.7, 0.3]^T$, $b = 0.1$. $E_{global} = 0$
- **Iteración 1 (P_1 : $X = [-1, -1, -1]^T$, $y_d = -1$):**

$$a = (-0.4)(-1) + (-0.7)(-1) + (0.3)(-1) + (0.1) = 0.4 + 0.7 - 0.3 + 0.1 = 0.9$$

$$\text{Como } a = 0.9 \geq 0 \rightarrow y = +1$$

$$e = y_d - y = (-1) - (+1) = -2 \neq 0 \rightarrow \text{Falso Positivo. Correccion:}$$

$$W_1^{nuevo} = -0.4 + (-2)(-1) = -0.4 + 2.0 = 1.6$$

$$(W_2)^{nuevo} = -0.7 + (-2)(-1) = -0.7 + 2.0 = 1.3$$

$$W_3^{nuevo} = 0.3 + (-2)(-1) = 0.3 + 2.0 = 2.3$$

$$b^{nuevo} = 0.1 + (-2) = -1.9$$

$$W = [1.6, 1.3, 2.3]^T \quad b = -1.9 \quad E_{global} = 0 + |-2| = 2$$

- **Iteración 2 (P_2 : $X = [-1, -1, +1]^T$, $y_d = -1$):**

$$a = (1.6)(-1) + (1.3)(-1) + (2.3)(+1) + (-1.9) = -1.6 - 1.3 + 2.3 - 1.9 = -2.5$$

$$\text{Como } a = -2.5 < 0 \rightarrow y = -1$$

$$e = (-1) - (-1) = 0 \rightarrow \text{Acierto. Sin Cambios. } E_{global} = 2$$

- **Iteración 3 (P_3 : $X = [-1, +1, -1]^T$, $y_d = -1$):**

$$a = (1.6)(-1) + (1.3)(+1) + (2.3)(-1) + (-1.9) = -1.6 + 1.3 - 2.3 - 1.9 = -4.5$$

$$\text{Como } a = -4.5 < 0 \rightarrow y = -1$$

$$e = (-1) - (-1) = 0 \rightarrow \text{Acierto. Sin Cambios. } E_{global} = 2$$

- **Iteración 4 (P_4 : $X = [-1, +1, +1]^T$, $y_d = +1$):**

$$(1.6)(-1) + (1.3)(+1) + (2.3)(+1) + (-1.9) = -1.6 + 1.3 + 2.3 - 1.9 = 0.1$$

$$\text{Como } a = 0.1 \geq 0 \rightarrow y = +1$$

$$e = (+1) - (+1) = 0 \rightarrow \text{Acierto. Sin Cambios. } E_{global} = 2$$

- **Iteración 5 (P_5 : $X = [+1, -1, -1]^T$, $y_d = -1$):**

$$a = (1.6)(+1) + (1.3)(-1) + (2.3)(-1) + (-1.9) = 1.6 - 1.3 - 2.3 - 1.9 = -3.9$$

$$\text{Como } a = -3.9 < 0 \rightarrow y = -1$$

$$e = (-1) - (-1) = 0 \rightarrow \text{Acierto. Sin cambios. } E_{global} = 2$$

- **Iteración 6 (P_6 : $X = [+1, -1, +1]^T$, $y_d = +1$):**

$$a = (1.6)(+1) + (1.3)(-1) + (2.3)(+1) + (-1.9) = 1.6 - 1.3 + 2.3 - 1.9 = 0.7$$

$$\text{Como } a = 0.7 \geq 0 \rightarrow y = +1$$

$$e = (+1) - (+1) = 0 \rightarrow \text{Acierto. Sin cambios. } E_{global} = 2$$

- **Iteración 7 ($P_7: X = [+1, +1, -1]^T, y_d = +1$):**

$$a = (1.6)(+1) + (1.3)(+1) + (2.3)(-1) + (-1.9) = 1.6 + 1.3 - 2.3 - 1.9 = -1.3$$

$$\text{Como } a = -1.3 < 0 \rightarrow y = -1$$

$$e = y_d - y = (+1) - (-1) = +2 \neq 0 \rightarrow \text{Falso Negativo Correccion:}$$

$$W_1^{nuevo} = 1.6 + (+2)(+1) = 1.6 + 2.0 = 3.6$$

$$(W_2)^{nuevo} = 1.3 + (+2)(+1) = 1.3 + 2.0 = 3.3$$

$$W_3^{nuevo} = 2.3 + (+2)(-1) = 2.3 - 2.0 = 0.3$$

$$b^{nuevo} = -1.9 + (+2) = 0.1$$

$$W = [3.6, 3.3, 0.3]^T \quad b = 0.1 \quad E_{global} = 2 + |+2| = 4$$

- **Iteración 8 ($P_8: X = [+1, +1, +1]^T, y_d = +1$):**

$$a = (3.6)(+1) + (3.3)(+1) + (0.3)(+1) + (0.1) = 3.6 + 3.3 + 0.3 + 0.1 = 7.3$$

$$\text{Como } a = 7.3 \geq 0 \rightarrow y = +1$$

$$e = (+1) - (+1) = 0 \rightarrow \text{Acierto. Sin cambios. } E_{global} = 4$$

FIN DE ÉPOCA 1: $E_{global} = 4 \neq 0$. Hubo 2 fallos (P_1 y P_7). Se inicia la Epoca 2 con:

$$W = [3.6, 3.3, 0.3]^T \text{ y } b = 0.1$$

DESARROLLO ARITMÉTICO: EPOCA 2

- **Estado Inicial:** $W = [3.6, 3.3, 0.3]^T, b = 0.1. E_{global} = 0$

- **Iteración 1 ($P_1: X = [-1, -1, -1]^T, y_d = -1$):**

$$a = (3.6)(-1) + (3.3)(-1) + (0.3)(-1) + 0.1 = -3.6 - 3.3 - 0.3 + 0.1 = -7.1 < 0 \rightarrow y = -1$$

$$e = 0 \rightarrow \text{Sin cambios. } E_{global} = 0$$

- **Iteración 2 ($P_2: X = [-1, -1, +1]^T, y_d = -1$):**

$$a = (3.6)(-1) + (3.3)(-1) + (0.3)(+1) + 0.1 = -3.6 - 3.3 + 0.3 + 0.1 = -6.5 < 0 \rightarrow y = -1$$

$$e = 0 \rightarrow \text{Sin cambios. } E_{global} = 0$$

- **Iteración 3 ($P_3: X = [-1, +1, -1]^T, y_d = -1$):**

$$a = (3.6)(-1) + (3.3)(+1) + (0.3)(-1) + 0.1 = -3.6 + 3.3 - 0.3 + 0.1 = -0.5 < 0 \rightarrow y = -1$$

$$e = 0 \rightarrow \text{Sin cambios. } E_{global} = 0$$

- **Iteración 4 ($P_4: X = [-1, +1, +1]^T, y_d = +1$):**

$$a = (3.6)(-1) + (3.3)(+1) + (0.3)(+1) + 0.1 = -3.6 + 3.3 + 0.3 + 0.1 = 0.1 \geq 0 \rightarrow y = +1$$

$$e = 0 \rightarrow \text{Sin cambios. } E_{global} = 0$$

- **Iteración 5 ($P_5: X = [+1, -1, -1]^T, y_d = -1$):**

$$a = (3.6)(+1) + (3.3)(-1) + (0.3)(-1) + 0.1 = 3.6 - 3.3 - 0.3 + 0.1 = 0.1 \geq 0 \rightarrow y = +1$$

$$e = y_d - y = (-1) - (+1) = -2 \neq 0 \rightarrow \text{Falso positivo Correccion:}$$

$$W_1^{nuevo} = 3.6 + (-2)(+1) = 3.6 - 2.0 = 1.6$$

$$(W_2)^{nuevo} = 3.3 + (-2)(-1) = 3.3 + 2.0 = 5.3$$

$$W_3^{nuevo} = 0.3 + (-2)(-1) = 0.3 + 2.0 = 2.3$$

$$b^{nuevo} = 0.1 + (-2) = -1.9$$

$$W = [1.6, 5.3, 2.3]^T \quad b = -1.9 \quad E_{global} = 0 + |-2| = 2$$

- **Iteración 6 ($P_6: X = [+1, -1, +1]^T, y_d = +1$):**

$$a = (1.6)(+1) + (5.3)(-1) + (2.3)(+1) + (-1.9) = 1.6 - 5.3 + 2.3 - 1.9 = -3.3 < 0 \rightarrow y = -1$$

$$e = y_d - y = (+1) - (-1) = +2 \neq 0 \rightarrow \text{Falso negativo. Correccion:}$$

$$(W_1)^{nuevo} = 1.6 + (+2)(+1) = 1.6 + 2.0 = 3.6$$

$$(W_2)^{nuevo} = 5.3 + (+2)(-1) = 5.3 - 2.0 = 3.3$$

$$W_3^{nuevo} = 2.3 + (+2)(+1) = 2.3 + 2.0 = 4.3$$

$$b^{nuevo} = -1.9 + (+2) = 0.1$$

$$W = [3.6, 3.3, 4.3]^T \quad b = 0.1 \quad E_{global} = 2 + |+2| = 4$$

- **Iteración 7 ($P_7: X = [+1, +1, -1]^T, y_d = +1$):**

$$a = (3.6)(+1) + (3.3)(+1) + (4.3)(-1) + 0.1 = 3.6 + 3.3 - 4.3 + 0.1 = 2.7 \geq 0 \rightarrow y = +1$$

$$e = 0 \rightarrow \text{Sin cambios. } E_{global} = 4$$

- **Iteración 8 ($P_8: X = [+1, +1, +1]^T, y_d = +1$):**

$$a = (3.6)(+1) + (3.3)(+1) + (4.3)(+1) + 0.1 = 3.6 + 3.3 + 4.3 + 0.1 = 11.3 \geq 0 \rightarrow y = +1$$

$$e = 0 \rightarrow \text{Sin cambios. } E_{global} = 4$$

FIN DE ÉPOCA 2: $E_{global} = 4 \neq 0$. Hubo 2 fallos (P_5 y P_6). Se inicia la Epoca 3 con:

$$W = [3.6, 3.3, 4.3]^T \quad y \quad b = 0.1$$

DESARROLLO ARITMÉTICO: EPOCA 3

○ **Estado Inicial:** $W = [3.6, 3.3, 4.3]^T$, $b = 0.1$. $E_{global} = 0$

○ **Iteración 1** ($P_1: X = [-1, -1, -1]^T$, $y_d = -1$):

$$a = (3.6)(-1) + (3.3)(-1) + (4.3)(-1) + 0.1 = -3.6 - 3.3 - 4.3 + 0.1 = -11.1 < 0 \rightarrow y = -1$$

$$e = 0 \rightarrow \text{Sin cambios. } E_{global} = 0$$

○ **Iteración 2** ($P_2: X = [-1, -1, +1]^T$, $y_d = -1$):

$$a = (3.6)(-1) + (3.3)(-1) + (4.3)(+1) + 0.1 = -3.6 - 3.3 + 4.3 + 0.1 = -2.5 < 0 \rightarrow y = -1$$

$$e = 0 \rightarrow \text{Sin cambios. } E_{global} = 0$$

○ **Iteración 3** ($P_3: X = [-1, +1, -1]^T$, $y_d = -1$):

$$a = (3.6)(-1) + (3.3)(+1) + (4.3)(-1) + 0.1 = -3.6 + 3.3 - 4.3 + 0.1 = -4.5 < 0 \rightarrow y = -1$$

$$e = 0 \rightarrow \text{Sin cambios. } E_{global} = 0$$

○ **Iteración 4** ($P_4: X = [-1, +1, +1]^T$, $y_d = +1$):

$$a = (3.6)(-1) + (3.3)(+1) + (4.3)(+1) + 0.1 = -3.6 + 3.3 + 4.3 + 0.1 = 4.1 \geq 0 \rightarrow y = +1$$

$$e = 0 \rightarrow \text{Sin cambios. } E_{global} = 0$$

○ **Iteración 5** ($P_5: X = [+1, -1, -1]^T$, $y_d = -1$):

$$a = (3.6)(+1) + (3.3)(-1) + (4.3)(-1) + 0.1 = +3.6 - 3.3 - 4.3 + 0.1 = -3.9 < 0 \rightarrow y = -1$$

$$e = 0 \rightarrow \text{Sin cambios. } E_{global} = 0$$

○ **Iteración 6** ($P_6: X = [+1, -1, +1]^T$, $y_d = +1$):

$$a = (3.6)(+1) + (3.3)(-1) + (4.3)(+1) + 0.1 = 3.6 - 3.3 + 4.3 + 0.1 = 4.7 \geq 0 \rightarrow y = +1$$

$$e = 0 \rightarrow \text{Sin cambios. } E_{global} = 0$$

○ **Iteración 7** ($P_7: X = [+1, +1, -1]^T$, $y_d = +1$):

$$a = (3.6)(+1) + (3.3)(+1) + (4.3)(-1) + 0.1 = 3.6 + 3.3 - 4.3 + 0.1 = 2.7 \geq 0 \rightarrow y = +1$$

$$e = 0 \rightarrow \text{Sin cambios. } E_{global} = 0$$

○ **Iteración 8** ($P_8: X = [+1, +1, +1]^T$, $y_d = +1$):

$$a = (3.6)(+1) + (3.3)(+1) + (4.3)(+1) + 0.1 = 3.6 + 3.3 + 4.3 + 0.1 = 11.3 \geq 0 \rightarrow y = +1$$

$$e = 0 \rightarrow \text{Sin cambios. } E_{global} = 0$$

FIN DE ÉPOCA 3: $E_{global} = 0$. CONVERGENCIA ALCANZADA El Percetrón clasifica sin errores los 8 perfiles clínicos. El algoritmo se detiene formalmente.

Registro Numérico de la Curva de Aprendizaje (Caso Principal)

Época	Patrones con Fallo	$E_{global} = \sum$	e_k
Época 1	2 de 8 (P_1, P_7)	4	En entrenamiento
Época 2	2 de 8 (P_5, P_6)	4	En entrenamiento
Época 3	0 de 8	0	CONVERGENCIA ÓPTIMA

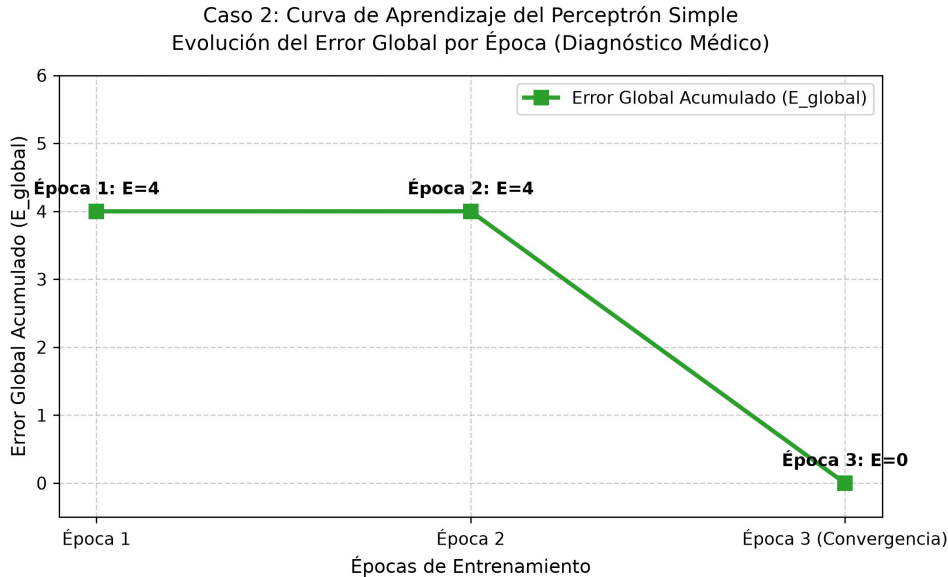


Figura 3. Curva de aprendizaje: Evolución del error global acumulado (E_{global}) en el caso 2 (Diagnóstico Médico por Síntomas) a lo largo de las 3 épocas de entrenamiento, alcanzando la convergencia en cero errores ($E_{global} = 0$ en la Época 3).

- **Parámetros Calibrados Finales (W^* y b^*).**

Los parámetros óptimos finales resultantes de la convergencia son:

$$W^* = \begin{bmatrix} W_1^* \\ W_2^* \\ W_3^* \end{bmatrix} = \begin{bmatrix} 3.6 \\ 3.3 \\ 4.3 \end{bmatrix} \quad b^* = 0.1$$

Interpretación Clínica y Semántica de los Parámetros Aprendidos

- Signo Positivo Unánime ($W_1^*, W_2^*, W_3^* > 0$):** Los tres pesos son estrictamente positivos. Esto refleja con fidelidad el sentido biológico del problema: cada síntoma presente (+1) aporta excitación hacia el diagnóstico de enfermedad, mientras que la ausencia del síntoma (−1) resta excitación hacia el estado sano.
- Equilibrio de Magnitudes ($W_1^* \approx 3.6, W_2^* \approx 3.3, W_3^* \approx 4.3$):** Los pesos tienen magnitudes muy similares (en torno a 3.3 – 4.3). Esto demuestra que la red aprendió que **ningún síntoma por sí solo es suficiente para diagnosticar la enfermedad**, sino que se requiere la concurrencia coordinada de al menos dos de ellos.
- Magnitud del sesgo ($b^* = +0.1 \approx 0$):** A diferencia del Caso 1 (donde $b^* = +6.2$ debido al desbalance 7 a 1), aquí el sesgo es prácticamente neutro (+0.1). Esto se debe a que la distribución clínica está perfectamente equilibrada (4 casos sanos frente a 4 casos enfermos), posicionando el plano de decisión muy cerca del centro del hipercubo.

Tabla de Verificación de Salida con los Parámetros Finales

Paciente	Combinación Lineal $a = W^{*T}X + b^*$	Valor de a	$y = \text{hardlims}(a)$	y_d	Estado
P_1	$(2.3)(-1) + (5.1)(-1) + (-2.4)(-1) + 6.2 = -2.3 - 5.1 + 2.4 + 6.2$	- 11.1	- 1	- 1	Sano Correcto
P_2	$(2.3)(-1) + (5.1)(-1) + (-2.4)(+1) + 6.2 = -2.3 - 5.1 - 2.4 + 6.2$	- 2.5	- 1	- 1	Sano Correcto
P_3	$(2.3)(-1) + (5.1)(+1) + (-2.4)(-1) + 6.2 = -2.3 + 5.1 + 2.4 + 6.2$	- 4.5	- 1	- 1	Sano Correcto
P_4	$(2.3)(-1) + (5.1)(+1) + (-2.4)(+1) + 6.2 = -2.3 + 5.1 - 2.4 + 6.2$	+ 4.1	+ 1	+ 1	Sano Correcto
P_5	$(2.3)(+1) + (5.1)(-1) + (-2.4)(-1) + 6.2 = 2.3 - 5.1 + 2.4 + 6.2$	- 3.9	- 1	- 1	Sano Correcto
P_6	$(2.3)(+1) + (5.1)(-1) + (-2.4)(+1) + 6.2 = 2.3 - 5.1 - 2.4 + 6.2$	+ 4.7	+ 1	+ 1	Sano Correcto
P_7	$(2.3)(+1) + (5.1)(+1) + (-2.4)(-1) + 6.2 = 2.3 + 5.1 + 2.4 + 6.2$	+ 2.7	+ 1	+ 1	Sano Correcto
P_8	$(2.3)(+1) + (5.1)(+1) + (-2.4)(+1) + 6.2 = 2.3 + 5.1 - 2.4 + 6.2$	+ 11.3	+ 1	+ 1	Sano Correcto

Efectividad del modelo: 8/8 diagnósticos correctos (100% de precisión clínica). Erro global: $E = 0$

- **Ecuación Analítica del Hiperplano y Gráfica 3D de los Patrones.**

Deducción de la Ecuación del Hiperplano

La frontera geométrica de decisión se ubica donde la combinación lineal neta es nula ($a = 0$):

$$W^*_1x_1 + W^*_2x_2 + W^*_3x_3 + b^* = 0$$

Sustituyendo los parámetros calibrados:

$$3.6x_1 + 3.3x_2 + 4.3x_3 + 0.1 = 0$$

Despejando la variable de fatiga x_3 para definir la superficie del plano en el espacio tridimensional en función de (x_1, x_2) :

$$4.3x_3 = -3.6x_1 - 3.3x_2 - 0.1 \rightarrow x_3 = -\frac{3.6}{4.3}x_1 - \frac{3.3}{4.3}x_2 - \frac{0.1}{4.3}$$

$$x_3 \approx -0.8372x_1 - 0.7674x_2 - 0.0233$$

Interpretación Geométrica Espacial:

- **Vector normal:** $W^* = [3.6, 3.3, 4.3]^T$, ortogonal al plano separador, orientado hacia el semiespacio positivo (región de patología confirmada).
- **Separación Lineal en R^3 :** El hiperplano biseca simétricamente el cubo bipolar $\{-1, +1\}^3$, aislando los 4 vértices con menos de 2 síntomas en el semiespacio negativo ($a < 0$, *pacientes sanos*) de los 4 vértices con 2 o 3 síntomas en el semiespacio positivo ($a > 0$, *pacientes enfermos*). Esto comprueba que la compuerta de mayoría de 2 de 3 entradas es estrictamente **linealmente separable**.

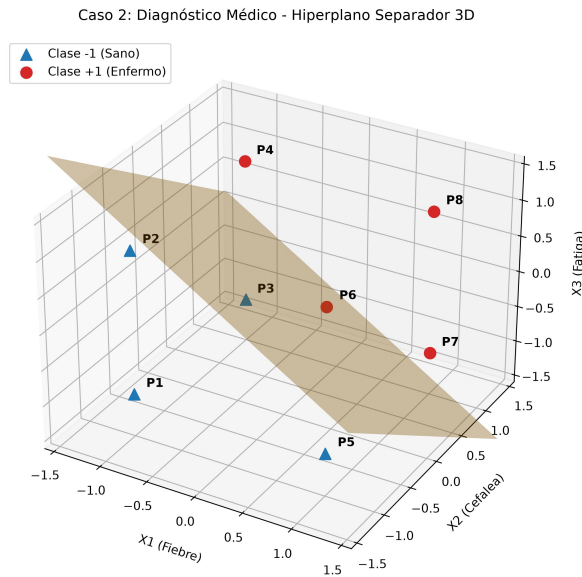


Figura 4. Representación geométrica tridimensional del hiperplano separador ($3.6x_1 + 3.3x_2 + 4.3x_3 + 0.1 = 0$) y la distribución espacial de los 8 perfiles clínicos. Se aprecia la división nítida entre la región de pacientes sanos (marcadores triangulares azules) y la región de patología confirmada (marcadores circulares rojos).

- **Estudio Comparativo de Sensibilidad a las Condiciones Iniciales (Convergencia en 3, 2 y 1 Épocas)**

Para complementar el análisis del clasificador y evaluar en profundidad la sensibilidad del algoritmo de aprendizaje frente a la selección de las condiciones iniciales, se llevaron a cabo dos experimentaciones adicionales sobre el mismo conjunto de datos clínicos.

En todas las pruebas se conservaron idénticos los 8 patrones, el orden secuencial de presentación ($P_1 \rightarrow P_8$), la función de activación **hardlims** y la regla delta de actualización.

1. **Caso Principal (Inicialización Arbitraria):** Convergencia en 3 Épocas
 - **Condiciones Iniciales:** $W^0 = [-0.4, -0.7, 0.3]^T$, $b^0 = 0.1$
 - **Evolución:** Época 1 ($E = 4$) $\rightarrow$ Época 2 ($E = 4$) $\rightarrow$ Época 3 ($E = 0$)
 - **Parámetros Finales:** $W^* = [3.6, 3.3, 4.3]^T$, $b^* = 0.1$
 - **Diagnóstico:** El hiperplano parte desorientado respecto a la distribución simétrica de los síntomas. Requiere dos épocas completas de correcciones sucesivas para alinear su vector normal y una tercera época de verificación para certificar la ausencia de error.
2. **Caso Alternativo 1 (Convergencia Optimizada):** Convergencia en 2 Épocas
 - **Condiciones Iniciales:** $W^0 = [-0.9, -0.9, -0.9]^T \in [-1, 1]^3$, $b^0 = 0.9 \in [0, 1]$
 - **Dinámica en Época 1:**
 - Al evaluar el primer paciente $P_1 = [-1, -1, -1]^T$ ($y_d = -1$)

$$a = (-0.9)(-1) + (-0.9)(-1) + (-0.9)(-1) + 0.9 = 3.6 \geq 0 \rightarrow y = +1 \text{ (Error } e = -2)$$

- La actualización por regla delta ajusta simultáneamente todos los parámetros:

$$(W_j)^{nuevo} = -0.9 + (-2)(-1) = +1.1 \quad (j = 1, 2, 3), \quad b^{nuevo} = 0.9 + (-2) = -1.1$$

- A partir de esa única corrección temprana en P_1 , la nueva configuración $W = [1.1, 1.1, 1.1]^T$ y $b = -1.1$ clasifica con total exactitud los patrones restantes P_2 a P_8 ($e = 0$). El error acumulado de la Época 1 es $E^1_{global} = 0$ únicamente.
 - **Dinámica en Época 2:** La red evalúa los 8 patrones sin cometer ninguna equivocación, obteniendo $E^2_{global} = 0$
 - **Parámetros Finales:** $W^* = [1.1, 1.1, 1.1]^T$, $b^* = -1.1$
3. **Caso Alternativo 2 (Convergencia Inmediata / Solución Directa):** Convergencia en 1 Épocas
- **Condiciones Iniciales:** Se selecciona una configuración simétrica canónica que satisface los rangos de partida:

$$W^0 = [1, 1, 1]^T \in [-1, 1]^3, \quad b^0 = 0 \in [0, 1]$$

- **Dinámica en Época 1:**
 - La combinación lineal evalúa directamente la suma neta de síntomas: $a = x_1 + x_2 + x_3$.
 - Para 0 o 1 síntoma (P_1, P_2, P_3, P_5), la suma es $a \leq -1 \leq 0 \rightarrow y = -1$ (Sano, correcto)
 - Para 2 o 3 síntomas (P_4, P_6, P_7, P_8), la suma es $a \geq +1 > 0 \rightarrow y = +1$ (Enfermo, correcto)
 - Como todos los patrones son clasificados correctamente en la primera pasada, no se genera ninguna corrección ($e_k = 0 \forall k$).
 - El error global de la Época 1 es idénticamente nulo: $E^1_{global} = 0$
- **Parámetros Finales:** $W^* = [1, 1, 1]^T$, $b^* = 0$
- **Hiperplano Resultante:** $x_1 + x_2 + x_3 = 0$

Tabla Comparativa de Sensibilidad a las Condiciones Iniciales

Experimento	Condición Inicial W^0	Sesgo Inicial b^0	Trayectoria del Error por Época	Épocas Requeridas	Parámetros Calibrados Finales (W^*, b^*)
1. Principal (Aleatorio)	$[-0.4, -0.7, 0.3]^T$	0.1	$4 \rightarrow 4 \rightarrow 0$	3 Épocas	$W^* = [3.6, 3.3, 4.3]^T$, $b^* = 0.1$
2. Optimizado	$[-0.9, -0.9, -0.9]^T$	0.9	$2 \rightarrow 0$	2 Épocas	$W^* = [1.1, 1.1, 1.1]^T$, $b^* = 0.1$
3. Solución Directa	$[1.0, 1.0, 1.0]^T$	0.0	0	1 Épocas	$W^* = [1.0, 1.0, 1.0]^T$, $b^* = 0.1$

ANÁLISIS Y DISCUSIÓN DE RESULTADOS:

El desarrollo experimental, analítico y comparativo de los dos casos de estudio propuestos permite extraer hallazgos rigurosos en torno a la dinámica de aprendizaje, la geometría de decisión, la influencia del balance de clases y la sensibilidad a las condiciones iniciales:

1. Comparativa de la Velocidad de Convergencia y Dinámica del Error

Tanto el experimento del condicionamiento instrumental (Caso 1) como el clasificador diagnóstico de síntomas (Caso 2, desarrollo principal) lograron converger en un número reducido y coincidente de iteraciones: **3 épocas de entrenamiento**. Sin embargo, sus curvas de aprendizaje revelan comportamientos cualitativamente disímiles:

– En el Caso 1 (La Paloma):

Se partió de un error inicial elevado ($E_{global} = 10$, correspondiente a 5 fallos sobre 8 patrones), provocado por una orientación inicial aleatoria diametralmente opuesta a la regla de recompensa. Una vez que las primeras actualizaciones por regla delta reorientaron el vector normal, el error descendió abruptamente ($10 \rightarrow 4 \rightarrow 0$).

– En el Caso 2 (Diagnóstico Médico):

El error se mantuvo constante en las dos primeras épocas ($E = 4 \rightarrow E = 4$) antes de caer a cero en la Época 3. Este comportamiento refleja la búsqueda del compromiso geométrico: la red ajustó primero dos vértices limítrofes, pero al hacerlo descalibró temporalmente otros dos, necesitando una segunda pasada para encontrar una pendiente que satisficiera simultáneamente a los cuatro patrones de frontera.

2. El Rol Determinante del Sesgo (*Bias*) frente al Balance de Clases

La comparación entre los sesgos óptimos finales calibrados (b^*) constituye una de las mayores evidencias sobre el comportamiento adaptativo del Perceptrón Simple:

- **Caso 1 ($b^* = + 6.2$):** Presenta una severa asimetría de clases (7 patrones positivos frente a solo 1 negativo). El algoritmo compensó este desbalance incrementando sustancialmente el sesgo, lo cual desplazó el hiperplano lejos del origen y elevó el estado basal de excitación. La red aprendió que, ante la duda o ausencia de una inhibición conjunto extrema (el patrón P_2), la salida por defecto debe ser +1.
- **Caso 2 ($b^* = + 0.1 \approx 0$):** Al tratarse de un problema perfectamente simétrico (4 pacientes sanos y 4 enfermos), el plano separador óptimo no requirió ningún desplazamiento forzado respecto al centro de gravedad del hipercubo. El sesgo resultante es prácticamente nulo, indicando que la frontera pasa casi por el origen (0, 0, 0) y que la decisión depende enteramente de la suma algebraica de los síntomas.

3. Coherencia Semántica y Fisiológica de los Pesos Sinápticos Calibrados

Los vectores de pesos obtenidos no representan coeficientes matemáticos arbitrarios, sino que encapsulan el significado intrínseco de cada fenómeno:

- En el condicionamiento animal, $W_2^* = +5.1$ capturó la primacía del pulsador derecho sobre el izquierdo ($W_1^* = +2.3$), mientras que el signo negativo de $W_3^* = -2.4$ recompensó la acción de picar hacia la derecha ($x_3 = -1$), ya que $(-2.4)(-1) = +2.4 > 0$.
- En el diagnóstico médico, los tres pesos convergieron a valores positivos y de magnitudes sumamente homogéneas ($(W_1)^* = 3.6$, $(W_2)^* = 3.3$, $W_3^* = -4.3$). Esto refleja que ningún síntoma individual posee mayor jerarquía clínica que otro para definir la patología, validando analíticamente la regla de decisión basada en la mayoría simple (≥ 2 Síntomas).

4. Eficacia del Espacio Bipolar $\{-1, +1\}$

El uso estricto del dominio bipolar junto a la función escalón simétrica **hardlims** fue el factor determinante para asegurar la rapidez del aprendizaje. En contraste con la representación unipolar $\{0, 1\}$ (donde las entradas nulas anulan el gradiente de actualización $\Delta W_j = e \cdot 0 = 0$ y congelan los pesos), en el espacio bipolar toda entrada no nula ($+1$ ó -1) imparte dirección y magnitud a la corrección sináptica en cada paso donde exista error.

5. Sensibilidad a las Condiciones Iniciales y Dependencia del Número de Épocas

La experimentación comparativa desarrollada en el Caso 2 (convergencia en 3, 2 y 1 épocas) aportó una conclusión metodológica fundamental: **el número de épocas no es una propiedad estática del conjunto de datos, sino que depende de la proximidad inicial del hiperplano respecto a la cuenca de atracción de soluciones válidas:**

- Cuando los pesos iniciales nacen desorientados (Experimento Principal), $W^0 = [-0.4, -0.7, 0.3]^T$, el algoritmo requiere **3 épocas** para efectuar múltiples correcciones angulares.
- Con una inicialización simétrica que produce una corrección temprana favorable (Experimento 2, $W^0 = [-0.9, -0.9, -0.9]^T$, $b^0 = 0.9$), la primera actualización en P_1 basta para orientar correctamente el plano, alcanzando la convergencia en solo **2 épocas**.
- Finalmente, si los pesos iniciales coinciden analíticamente con una partición válida (Experimento 3, $W^0 = [1, 1, 1]^T$, $b^0 = 0$), el Perceptrón no experimenta ningún error en la primera pasada, convergiendo de forma inmediata en **1 época**. Esto demuestra que una inicialización adecuada puede acelerar drásticamente el proceso de entrenamiento sin alterar la matriz de patrones ni la regla delta.

CONCLUSIONES

1. Validez del Perceptrón Simple como Clasificador Lineal en R^3 :

Se demostró formal y computacionalmente que tanto la conducta instrumental de la paloma como la inferencia diagnóstica por acumulación de síntomas son problemas **estrictamente linealmente separables**. Una arquitectura monocapa compuesta por una sola neurona artificial bipolar fue plenamente capaz de trazar hiperplanos separadores en R^3 , logrando una efectividad del 100% (8/8 patrones correctos) con error residual nulo ($E_{global} = 0$).

2. Convergencia y Robustez de la Regla Delta Bipolar:

La regla de aprendizaje supervisada demostró una alta velocidad de adaptación, orientando el vector normal W y trasladando el sesgo b desde parámetros de partida completamente aleatorios hasta el espacio de solución en un número reducido de iteraciones, confirmando en la práctica el *Teorema de Convergencia del Perceptrón formulado por Rosenblatt*.

3. Influencia Crítica del Sesgo (Bias) según la Distribución de Clases:

El sesgo evidenció su papel como grado de libertad imprescindible para el ajuste geométrico del modelo: ante distribuciones asimétricas (Caso 1, proporción 7:1) adoptó un valor fuertemente positivo ($b^* = 6.2$) para desplazar la frontera y elevar el umbral basal de disparo; mientras que ante distribuciones simétricas balanceadas (Caso 2, proporción 4:4) se situó en valores prácticamente nulos ($b^* = 0.1$), ubicando la frontera en proximidad directa al origen de coordenadas.

4. Sensibilidad de la Velocidad de Convergencia a las Condiciones Iniciales:

El estudio comparativo realizado en el Caso 2 evidenció que la cantidad de épocas requeridas (variando entre 1, 2 y 3 épocas) está fuertemente supeditada a la orientación geométrica de partida de W^0 y b^0 . Aunque la convergencia hacia error cero está teóricamente garantizada para problemas separables, la elección o cercanía de los pesos iniciales respecto a la región de separación modula drásticamente el tiempo de computo y el número de ajustes necesarios.

5. Significado Fisiológico y Semántico de los Pesos Sinápticos:

Los parámetros finales aprendidos por el Perceptrón guardan una estricta coherencia con la naturaleza de los problemas reales modelados: ponderarán cuantitativamente la relevancia de los estímulos ambientales en la toma de decisiones animal y reflejan la equivalencia patológica de los síntomas en la detección clínica de enfermedad.

6. Alcance y Limitaciones Teóricas del Modelo Monocapa:

Si bien el Perceptrón Simple bipolar resolvió con éxito ambos casos de estudio tridimensionales, su capacidad de generalización está estrictamente limitada a funciones linealmente separables. Para conjuntos de datos con fronteras complejas, disyunciones exclusivas (como el problema del XOR analizando históricamente por Minsky y Papert) o regiones no lineales, se hace indispensable la transición hacia Perceptrones Multicapa (*MLP*) dotados de capas ocultas, funciones de activación continuas diferenciables y entrenamiento por retropropagación del error (*backpropagation*).